

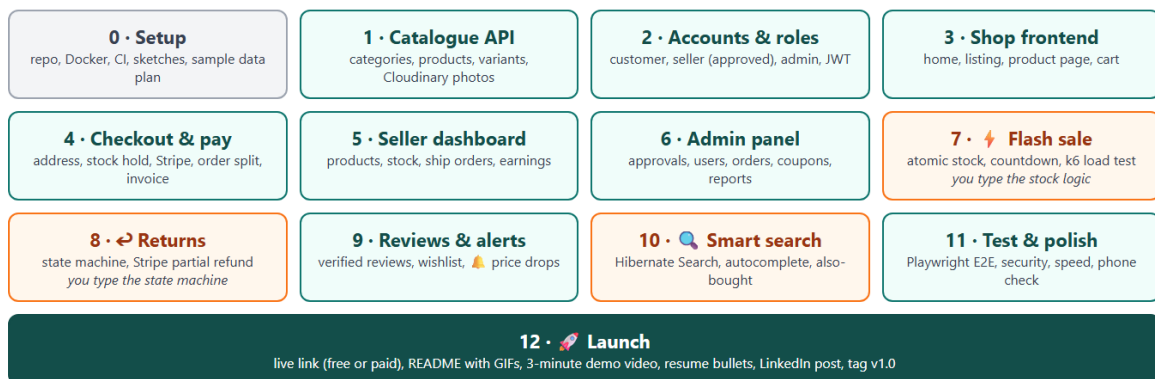
TriVoKo

Multi-seller E-commerce Marketplace

Phase Plan

Step-by-step checklists for the 13 phases: who does each step, what you get, how you know it is done, and the questions to explain it back

The 13 phases (each phase ends with a working, tested, pushed app)



Must-have core = phases 0-8 (a complete marketplace with the two strongest standouts). Phases 9-10 make it shine. If time runs short, 9 and 10 can be shortened without breaking anything.

Madhavan Suresh

Java Full Stack main portfolio project · React + Spring Boot + MySQL + Stripe

Version 1.0 · 3 October 2026 · companion to the Master Plan

Contents

How to use this plan	3
1.1 The 13 phases	3
1.2 Who does each step.....	4
1.3 The rules for every phase.....	4
1.4 Where things live	4
Phase 0 - Setup.....	6
Steps.....	6
Output of this phase	6
Done when.....	7
Explain it back	7
Phase 1 - Catalogue API	8
Steps.....	8
Output of this phase	8
Done when.....	9
Explain it back	9
Phase 2 - Accounts & roles.....	10
Steps.....	10
Output of this phase	10
Done when.....	10
Explain it back	10
Phase 3 - Shop frontend.....	12
Steps.....	12
Output of this phase	12
Done when.....	12
Explain it back	12
Phase 4 - Checkout & payments	14
Steps.....	14
Output of this phase	14
Done when.....	15
Explain it back	15
Phase 5 - Seller dashboard.....	16
Steps.....	16
Output of this phase	16
Done when.....	16
Explain it back	16
Phase 6 - Admin panel	18

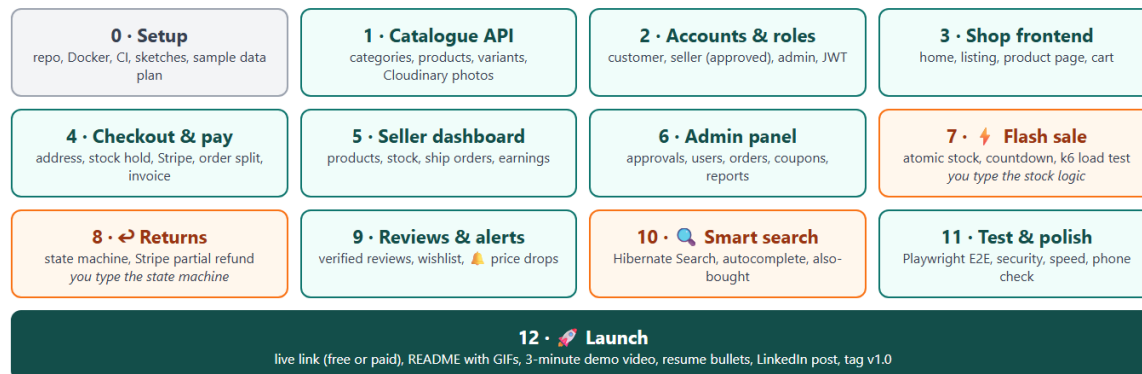
Steps.....	18
Output of this phase	18
Done when.....	18
Explain it back	18
Phase 7 - Flash sale	19
Steps.....	19
Output of this phase	19
Done when.....	19
Explain it back	20
Phase 8 - Returns & refunds	21
Steps.....	21
Output of this phase	21
Done when.....	21
Explain it back	21
Phase 9 - Reviews, wishlist, alerts.....	23
Steps.....	23
Output of this phase	23
Done when.....	23
Explain it back	23
Phase 10 - Smart search.....	24
Steps.....	24
Output of this phase	24
Done when.....	24
Explain it back	24
Phase 11 - Testing & polish	25
Steps.....	25
Output of this phase	25
Done when.....	25
Explain it back	25
Phase 12 - Launch	26
Steps.....	26
Output of this phase	26
Done when.....	26
Explain it back	26

How to use this plan

This is the **step-by-step plan** for building TriVoKo. The **Master Plan** (1 October 2026) explains *what* we build and *why*. This document says *in which order* and *how you know a phase is finished*.

Real-life example: A building plan has two papers: the architect's drawing (what the house looks like) and the contractor's schedule (foundation first, then walls, then roof). The Master Plan is the drawing. This Phase Plan is the schedule.

The 13 phases (each phase ends with a working, tested, pushed app)



Must-have core = phases 0-8 (a complete marketplace with the two strongest standouts). Phases 9-10 make it shine. If time runs short, 9 and 10 can be shortened without breaking anything.

The 13 phases

1.1 The 13 phases

Phase	Name	Goal	Time	Group
0	Setup	Project ready to build	3-4 days	Core
1	Catalogue API	Products exist	1 week	Core
2	Accounts & roles	Safe login for 3 roles	1 week	Core
3	Shop frontend	Customer can shop (no payment yet)	1.5 weeks	Core
4	Checkout & payments	Real orders, paid once, split per seller	2 weeks	Core
5	Seller dashboard	Sellers run their shop	1.5 weeks	Core
6	Admin panel	Admin controls the marketplace	1 week	Core
7	Flash sale	Standout #1 - never oversells	1.5 weeks	Core
8	Returns & refunds	Standout #2 - exact partial refunds	1.5 weeks	Core
9	Reviews, wishlist, alerts	Standout #3 - alerts without spam	1 week	Shine
10	Smart search	Standout #4 - typo-tolerant search	1 week	Shine
11	Testing & polish	Portfolio quality	1 week	Always
12	Launch	Live and shown	1 week	Always

Total: about **15-16 weeks** at 2 hours per day. **Core** (Phases 0-8) is a complete marketplace with the two strongest standouts. **Shine** (9-10) can be made smaller if time is short. **Always** (11-12) is never skipped - a portfolio project that is not tested and not live is not finished.

1.2 Who does each step

Every step has a **Who** column:

Who	Meaning
Claude	Claude builds it and explains it line by line. You read, run it, and ask for changes.
★ Madhavan	You type the key logic. Claude explains the idea first with a picture, gives you the method signature and the test, then reviews what you wrote. These are the parts interviewers will ask about.
You	Something only you can do: create an account (Stripe, Cloudinary, GitHub), approve a sketch, choose hosting.
Together	We do it side by side - for example the real Stripe test or the demo video.

The three ★ parts are: the **order split** (Phase 4), the **flash-sale claim** (Phase 7) and the **return state machine** (Phase 8). For each one you also write at least one test.

1.3 The rules for every phase

1. Open the phase folder (**Phase_N_Name**) and its checklist document. Tick each step when it is done.
2. Do not start the next phase until **every "Done when" check passes** - shown working in the browser, not only in tests.
3. Every phase ends with the **finish routine**:
 - All tests pass on the laptop and **CI is green** on GitHub
 - Code committed and pushed with clear messages
 - The "Done when" checks shown working
 - **Explain it back**: you answer the questions at the end of the phase in your own words (spoken or written in "My notes")
 - Session summary + SESSION_LOG entry written

Note: A phase may be split over many days. The checklist document is the place where you write the start date, the finish date and your own notes.

1.4 Where things live

Folder	What is inside
00_Project_Documents\	Master Plan, this Phase Plan, guides, session history
Phase_0_Setup\ ... Phase_12_Launch\	One checklist document per phase (tick boxes + notes)
backend\	Spring Boot code - one project that grows every phase
frontend\	React code - one project that grows every phase
load-tests\, e2e\	k6 scripts (Phase 7), Playwright tests (Phase 11)

Folder	What is inside
docs\, tools\	Sketches, decisions, README diagrams; document generators, demo scripts

Phase 0 - Setup

Time: 3-4 days · **Group:** Core · **Folder:** `Phase_0_Setup\`

Goal: Create the TriVoKo project so that the backend, the frontend and the database all run on your laptop and on GitHub, and agree on the look of the shop before any page is coded.

Real-life example: Before a new shop opens in a mall, the owner gets the keys, puts up the name board, paints the walls in the brand colours and draws where every shelf will go. No goods are sold yet - but everything is ready.

Steps

#	Step	Who
1	Approve this Phase Plan (or ask for changes)	You
2	Turn <code>03_TriVoKo_Main_Project</code> into a Git repository: folders <code>backend/</code> , <code>frontend/</code> , <code>load-tests/</code> , <code>e2e/</code> , <code>docs/</code> , <code>tools/</code> ; <code>.gitignore</code> , <code>.gitattributes</code> , README	Claude
3	Spring Boot 4 + Java 25 skeleton, package <code>com.trivoko</code> , same versions as EventHub; dev and prod profiles; <code>.env</code> (git-ignored) for passwords	Claude
4	<code>docker-compose.yml</code> : MySQL 8.4 and Mailpit, with new ports (3307, 8026) so EventHub and TriVoKo never clash	Claude
5	Copy the EventHub base: ProblemDetail error handler, <code>/api/health</code> , Swagger, base SecurityConfig, Flyway, Testcontainers MySQL for tests	Claude
6	React 19 + Vite skeleton: Tailwind v4 with TriVoKo teal + saffron colour tokens, dark mode, React Router, Redux Toolkit, Axios, Vite proxy; footer shows "Backend: UP"	Claude
7	Copy the EventHub UI kit (Button, Card, Badge, Modal, Loader, FormField, Toaster) and recolour it; <code>/style-guide</code> page	Claude
8	TriVoKo logo (SVG) + favicon	Claude
9	GitHub Actions CI: backend build + tests, frontend lint + build + Vitest	Claude
10	Create the public GitHub repository <code>trivoko</code> (or say yes and Claude creates it) and push	You
11	Library check (small spikes): Hibernate Search 8, Stripe Java, Cloudinary, OpenPDF and Testcontainers all work with Spring Boot 4 - results written in <code>docs/decisions.md</code>	Claude
12	Sketches of the 6 key pages: Home, Search/listing, Product, Cart, Checkout, Seller dashboard	Claude
13	Sample-data plan: 10 categories, 8 sellers, 120 products with variants; where product photos come from	Claude
14	Approve the sketches and the sample-data plan (or ask for changes)	You

Output of this phase

- Spring Boot app at localhost:8080 with Swagger
- React app at localhost:5173 in TriVoKo colours
- MySQL + Mailpit in Docker
- GitHub repository `trivoko` with a green CI tick

- Approved sketches and sample-data plan

Done when

- The React page shows "Backend: UP"
- `docker compose up -d` starts MySQL and Mailpit with one command
- CI is green on GitHub
- Madhavan has approved the sketches

Explain it back

- What is a modular monolith, and why did we not choose microservices?
- Why does the database run in Docker instead of being installed on Windows?
- What does CI do on every push?

Skills you practise: Project setup, Maven, npm, Docker, environment variables, CI/CD, design tokens

Where your work goes: `backend\`, `frontend\`, `docker-compose.yml`, `.github\workflows\`, `docs\`

Phase 1 - Catalogue API

Time: 1 week · **Group:** Core · **Folder:** `Phase_1_Catalogue_API\`

Goal: Products exist in the database and anyone can browse them through the API: categories, filters, sorting, paging and product details.

Real-life example: The shelves are filled and every product has a price tag and a label. Customers can walk the aisles, but the billing counter is not open yet.

Steps

#	Step	Who
1	Flyway V1: <code>categories</code> (tree with parent), <code>sellers</code> (shop name, slug, city, status), <code>products</code> (status DRAFT / PENDING / ACTIVE / REJECTED / BLOCKED), <code>product_variants</code> (size, colour, price, MRP, stock, version), <code>product_images</code> , <code>price_history</code>	Claude
2	Flyway V2 seed data from the sample-data plan: 10 categories, 8 sellers, 120 products with variants and photos	Claude
3	Entities and repositories in modules <code>catalog</code> and <code>seller</code> ; money as <code>BigDecimal / DECIMAL(12,2)</code>	Claude
4	ArchUnit test for the module rule (a module may call another module's service, never its repository) - runs in CI from now on	Claude
5	DTO records + mapper; shared <code>PageResponse</code>	Claude
6	<code>GET /api/products</code> with filters (category incl. sub-categories, brand, min/max price, in stock), sort (price, newest) and paging (max 48)	Claude
7	<code>GET /api/products/{slug}</code> (variants, images, seller), <code>GET /api/categories</code> (tree), <code>GET /api/sellers/{slug}</code> (shop page)	Claude
8	Price-change service method that always writes <code>price_history</code> (used by price alerts in Phase 9)	Claude
9	Create a free Cloudinary account and put the 3 keys in <code>.env</code>	You
10	Cloudinary signed upload: <code>POST /api/uploads/signature</code> (images only, max 5 MB) - tried with one real photo	Claude
11	Tests: repository tests, service rules, every endpoint (Testcontainers MySQL)	Claude
12	Postman collection with checks, run with newman	Claude

Output of this phase

- Catalogue REST API in Swagger
- 120 sample products in MySQL
- Postman collection
- ArchUnit module rule in CI

Done when

- `GET /api/products?category=phones&sort=price` returns the correct pages
- All Postman checks pass
- CI green

Explain it back

- Why does stock live on the variant and not on the product?
- What is a Flyway migration and why do we never change the database by hand?
- How do filters become one SQL query (Specifications)?

Skills you practise: JPA entities, relationships, Flyway, Specifications, paging, DTOs, testing with Testcontainers

Where your work goes: `backend\src\main\java\com\trivoko\catalog, ...\seller, db\migration, postman\`

Phase 2 - Accounts & roles

Time: 1 week · **Group:** Core · **Folder:** `Phase_2_Accounts_and_Roles\`

Goal: Safe login for customers, sellers and admins; a customer can apply to become a seller; a seller can only touch their own products.

Real-life example: Every person in the mall gets the right card: shoppers can enter the shop floor, shopkeepers can enter only their own storeroom, and the mall office can open every door.

Steps

#	Step	Who
1	Flyway: <code>users</code> , <code>user_roles</code> , <code>addresses</code> ; <code>sellers.user_id</code> ; seller status PENDING / APPROVED / REJECTED / BLOCKED	Claude
2	Copy EventHub auth: JWT in httpOnly cookie, CSRF double-submit, BCrypt, 5 wrong logins = 15-minute lock; <code>/api/auth/register</code> , <code>/login</code> , <code>/logout</code> , <code>/me</code>	Claude
3	Roles CUSTOMER / SELLER / ADMIN; every new account is a CUSTOMER	Claude
4	Addresses API (max 5, one default)	Claude
5	"Become a seller": <code>POST /api/seller/apply</code> (shop name, city, optional GST) → PENDING; admin approve / reject endpoints (the admin pages come in Phase 6)	Claude
6	Seller products API: create (DRAFT), edit, submit for approval; <code>SellerAccess</code> bean checks ownership on every call	Claude
7	Audit log table + service (who changed what and when)	Claude
8	Dev demo accounts (admin, customers Ravi and Kavya, sellers Chennai Mobiles and Kovai Sports) with the password from <code>.env</code>	Claude
9	Tests: Seller A cannot read or edit Seller B's product (403/404), unapproved seller cannot sell, login lock, CSRF	Claude

Output of this phase

- Working register / login / logout with cookies
- Seller application and approval API
- Ownership checks with tests

Done when

- A test proves a seller cannot touch another seller's product (403)
- An unapproved seller cannot create products
- CI green

Explain it back

- Why is the JWT in an httpOnly cookie and not in localStorage?

- What does the CSRF token protect against?
- How does the server know a product belongs to the logged-in seller?

Skills you practise: Spring Security, JWT, cookies, CSRF, method security, ownership checks

Where your work goes: `backend\...\user`, `...\auth`, `...\seller`, `...\admin`

Phase 3 - Shop frontend

Time: 1.5 weeks · **Group:** Core · **Folder:** Phase_3_Shop_Frontend\

Goal: A customer can browse, search the catalogue, choose a size or colour and fill a cart from many sellers - on laptop and phone, light and dark. No payment yet.

Real-life example: The shop doors open and people can walk around and fill their trolleys. The billing counter still has a "coming soon" board.

Steps

#	Step	Who
1	Layout: navbar (search box, category menu, cart count, account menu), footer, mobile menu, Toaster	Claude
2	Home: deals, category tiles, new arrivals, flash-sale banner place	Claude
3	Listing page: filters and sort kept in the URL, pagination, product cards, skeleton loaders, empty state	Claude
4	Product page: photo gallery, variant picker (sold-out sizes disabled), price + MRP + % off, seller box, add to cart	Claude
5	Seller shop page	Claude
6	Cart backend: <code>cards</code> , <code>card_items</code> ; <code>GET/PUT/DELETE /api/cart/items</code> , <code>POST /api/cart/merge</code> (quantity capped by stock, max 10)	Claude
7	Cart frontend: guest cart in localStorage + Redux; cart page grouped by seller with the shipping rule; merge after login	Claude
8	Login, Register, Become-a-seller pages; route guards (copied from EventHub)	Claude
9	Addresses page	Claude
10	Vitest: cart maths, URL filters	Claude
11	Automated phone (375 px) + dark-mode scan of every page; fix what it finds	Claude

Output of this phase

- The shop website on real API data
- Cart that survives login
- Every page works on phone and in dark mode

Done when

- A guest adds 3 items from 2 sellers, logs in, and the cart is kept
- Every page passes the phone + dark-mode check

Explain it back

- Why are filters kept in the URL?

- What happens to the guest cart when you log in?
- Why does the server recalculate prices even though the cart shows them?

Skills you practise: React components, React Router, Redux Toolkit, forms, responsive Tailwind, localStorage

Where your work goes: `frontend\src\pages, ...\components, ...\store; backend\...\cart`

Phase 4 - Checkout & payments

Time: 2 weeks · **Group:** Core · **Folder:** `Phase_4_Checkout_and_Payments\`

Goal: One cart, one Stripe payment, one order split into one package per seller - with stock held safely, coupons, emails and a PDF invoice.

Real-life example: At a food court you pay once at the main counter, and your token is sent to the dosa stall and the juice stall separately. Each stall only sees its own part of your order.

Steps

#	Step	Who
1	Flyway: <code>orders</code> , <code>packages</code> , <code>order_items</code> (copy title, variant and price), <code>payments</code> , <code>processed_payment_events</code> , <code>coupons</code> , <code>coupon_redemptions</code>	Claude
2	<code>Money</code> helpers (rupees ↔ paise, rounding) with unit tests	Claude
3	Coupon rules: flat or %, minimum order, expiry, one use per customer; admin create endpoint	Claude
4	★ You type the ORDER SPLIT: group cart lines by seller into packages, shipping fee per package (free above ₹499, else ₹40), coupon share per item (by price, rounding remainder on the last item)	★ Madhavan
5	★ You write at least one test for the order split (2 sellers + coupon → exact amounts)	★ Madhavan
6	CheckoutService: recheck products, sellers, coupon and prices; hold stock with <code>UPDATE ... SET stock = stock - :qty WHERE stock >= :qty</code> ; roll back and name the item that ran out	Claude
7	Copy the EventHub PaymentGateway switch (Fake when no Stripe key, Stripe Checkout when set)	Claude
8	Stripe webhook (signature check) + "verify on return" fallback + idempotency → order PAID, packages PLACED, coupon used, cart emptied	Claude
9	Expiry job: unpaid after 10 minutes → EXPIRED, stock back; late payment re-takes stock or is refunded automatically	Claude
10	<code>notify</code> module (copy from EventHub): bell + emails after commit with retry; order email to customer, new-package email to each seller	Claude
11	PDF invoice with OpenPDF: <code>GET /api/orders/{number}/invoice.pdf</code>	Claude
12	Frontend: checkout page (address, coupon, summary per package, "price changed" notice), payment result, My orders, order detail	Claude
13	Concurrency test: 100 threads try to buy 10 units → exactly 10 orders	Claude
14	Create the Stripe test account, put <code>sk_test</code> in <code>.env</code> , install the Stripe CLI	You
15	Real Stripe test: pay with 4242 4242 4242 4242 and receive a real signed webhook (the EventHub to-do)	Together

Output of this phase

- Working checkout with Stripe test mode
- Split orders with packages per seller
- Order emails in Mailpit and PDF invoices

- Your own OrderSplitter class + test

Done when

- Pay once for items from 2 sellers → 2 packages PLACED
- The 100-thread stock test passes
- A real Stripe 4242 payment confirmed by a signed webhook

Explain it back

- Walk through checkout from "Place order" to "PLACED" - which table changes at each step?
- How is the coupon shared between packages, and why store it on each item?
- Why can a webhook arrive twice, and what stops a double order?

Skills you practise: Transactions, atomic SQL updates, money maths, Stripe Checkout, webhooks, idempotency, scheduled jobs, PDF

Where your work goes: `backend\...\order`, `...\payment`, `...\coupon`, `...\notify`; checkout pages in `frontend`

Phase 5 - Seller dashboard

Time: 1.5 weeks · **Group:** Core · **Folder:** `Phase_5_Seller_Dashboard\`

Goal: Sellers run their own shop: add products with photos, ship their packages, and see what they earn.

Real-life example: Each shopkeeper gets their own back office: a stock register, a packing table, and an account book that says how much the mall owes them.

Steps

#	Step	Who
1	Seller layout + dashboard: today's packages, low stock, earnings, products waiting for approval	Claude
2	Products list + product editor: variants table, photo upload to Cloudinary (drag to reorder, max 8, 5 MB), submit for approval; price change writes <code>price_history</code>	Claude
3	Low-stock warnings (default below 5)	Claude
4	<code>PackageStatus</code> enum with <code>allowedNext()</code> + <code>@Version</code> ; Packed → Shipped (tracking number) → Delivered (demo button)	Claude
5	Cancel before shipping (customer or seller): refund that package, stock back; race test cancel-vs-ship → one wins, other gets 409	Claude
6	<code>seller_ledger</code> : DELIVERED adds +90%; earnings page with the ledger lines	Claude
7	Customer order detail: tracking timeline per package; email + bell at every step	Claude
8	Sales charts for 7 / 30 / 90 days (Recharts, cached)	Claude
9	Tests: ownership, every package move, ledger amounts	Claude

Output of this phase

- Seller back office
- Package tracking timelines
- Earnings ledger and sales charts

Done when

- A seller ships a package and the customer sees the timeline update
- Cancel-vs-ship race test passes

Explain it back

- What is a state machine and why use `allowedNext()`?
- What does `@Version` do when two people change the same package?
- Why is the seller balance the sum of ledger lines instead of one number?

Skills you practise: State machines, optimistic locking, file upload, charts, caching

Where your work goes: `frontend\src\pages\seller`, `backend\...\order`, `...\seller`

Phase 6 - Admin panel

Time: 1 week · **Group:** Core · **Folder:** Phase_6_Admin_Panel\

Goal: The admin controls the marketplace: approves sellers and products, blocks bad actors, creates coupons and reads reports.

Real-life example: The mall office: it decides which shops may open, can close a shop that cheats customers, prints discount coupons and reads the daily sales register.

Steps

#	Step	Who
1	Admin layout + overview cards	Claude
2	Seller approvals (approve / reject with reason, email to the seller)	Claude
3	Product approvals (approve / reject with reason)	Claude
4	Search users, sellers and orders with paging	Claude
5	Block a seller → their products disappear from the shop; block a user → they cannot log in	Claude
6	Coupons page (create, edit, end)	Claude
7	Reports: sales per day, top products, top sellers, return rate (filled in Phase 8)	Claude
8	Activity log page from the audit log	Claude
9	Tests for every admin action and that non-admins get 403	Claude

Output of this phase

- Admin panel with approvals, users, coupons, reports, activity log

Done when

- Admin blocks a seller → their products disappear from the shop
- CI green

Explain it back

- How is "blocked" applied to every shop query without forgetting one?
- What is written to the audit log and why?

Skills you practise: Role-based pages, reporting queries (GROUP BY), audit trails

Where your work goes: frontend\src\pages\admin, backend\...\admin, ...\analytics

Phase 7 - Flash sale

Time: 1.5 weeks · **Group:** Core · **Folder:** `Phase_7_Flash_Sale\`

Goal: Standout #1: a flash sale where thousands click at once, exactly the right number win, and stock never goes below zero - proven with a 5,000-user load test.

Real-life example: A big sale at a shop with 100 TVs and 5,000 people at the door. A guard gives out exactly 100 tokens, one per person. Token 101 does not exist.

Steps

#	Step	Who
1	Flyway: <code>flash_sales</code> , <code>flash_sale_items</code> , <code>flash_sale_claims</code> with <code>UNIQUE(sale item, customer)</code>	Claude
2	Admin schedules a sale; sellers offer a variant with sale price + quantity; the quantity is taken out of normal stock	Claude
3	★ You type the CLAIM: one atomic <code>UPDATE ... SET sold = sold + 1 WHERE sold < quantity AND NOW() BETWEEN start AND end</code> , the unique claim row, and the 10-minute hold at the sale price	★ Madhavan
4	★ You type the GIVE-BACK: an unpaid claim returns its unit (<code>sold = sold - 1</code>)	★ Madhavan
5	Sale end: unsold units go back to normal stock	Claude
6	Rate limit on claim per user + IP (429 + Retry-After)	Claude
7	Frontend: home banner countdown (server time), sale page, live "N left" counter, instant "Sold out"	Claude
8	★ 1,000 threads / 100 units test - you write the final assertions	★ Madhavan
9	k6 load test: 5,000 virtual users against the app in Docker; record requests per second, p95, winners and errors	Claude
10	Decision: if p95 is above 500 ms, try Redis <code>DECR</code> and measure again; write the decision with both graphs	Together

Output of this phase

- Flash sale feature with countdown and live counter
- Your claim + give-back code
- k6 report and graph in `load-tests\results`

Done when

- k6 report: exactly 100 winners, 0 oversell, 0 errors
- p95 written in the README

Explain it back

- Why is "read stock, then write stock" wrong, and why does one UPDATE fix it?
- What stops one person from winning twice?
- What did k6 measure, and did we need Redis?

Skills you practise: Concurrency, atomic SQL, unique constraints, rate limiting, load testing with k6

Where your work goes: `backend\...\flashsale`, `load-tests\`, flash-sale pages in `frontend`

Phase 8 - Returns & refunds

Time: 1.5 weeks · **Group:** Core · **Folder:** `Phase_8>Returns_and_Refunds\`

Goal: Standout #2: a customer returns one item and gets back exactly the right money through a Stripe partial refund, while stock and the seller ledger stay correct.

Real-life example: You return one shirt from a bill of three. The shop checks it, takes it back to the shelf, and returns only that shirt's money - minus its part of the discount you used.

Steps

#	Step	Who
1	Flyway: <code>return_requests</code> , <code>refunds</code>	Claude
2	★ You type the RETURN STATE MACHINE: <code>ReturnStatus</code> enum with <code>allowedNext()</code> and the service method that moves a return	★ Madhavan
3	★ You write tests for allowed and illegal moves	★ Madhavan
4	Request a return: DELIVERED items only, within 7 days, once per item, reason + optional photo	Claude
5	Seller approve / reject with reason; auto-approve job after 2 days	Claude
6	Escalate once; admin makes the final decision	Claude
7	Picked up → Stripe partial refund of (price × qty – coupon share), one idempotency key per return	Claude
8	Refund done → stock back, minus line in the seller ledger, email + bell	Claude
9	Frontend: return form + timeline (customer), returns page (seller), disputes page (admin); return rate in admin reports	Claude
10	Tests: refund maths, double click = one refund	Claude

Output of this phase

- Complete returns flow for 3 roles
- Stripe partial refunds
- Your `ReturnStatus` state machine + tests

Done when

- Return one item of a 2-item order → the exact partial refund appears in the Stripe test dashboard
- Ledger and stock match

Explain it back

- Draw the return states and say who can make each move

- How is the refund amount calculated?
- Why can a refund never happen twice?

Skills you practise: State machines, Stripe refunds, idempotency, scheduled jobs

Where your work goes: `backend\...\returns`, `...\payment`, return pages in `frontend`

Phase 9 - Reviews, wishlist, alerts

Time: 1 week · **Group:** Shine · **Folder:** Phase_9_Reviews_Wishlist_Alerts\

Goal: Standout #3: verified-buyer reviews, a wishlist, and price-drop alerts that are useful and never spammy.

Real-life example: You ask the shopkeeper, "Call me if this TV gets cheaper." He calls once when it does - not five times when he changes the tag five times in a minute.

Steps

#	Step	Who
1	Flyway: <code>reviews</code> , <code>wishlist_items</code> (price when added, last alerted price)	Claude
2	Reviews only for delivered items, one per item; rating average and star breakdown on the product page	Claude
3	Wishlist API + heart button + wishlist page	Claude
4	<code>PriceChanged</code> event; alert job every 10 minutes using the latest price; one alert per drop; email + bell	Claude
5	"Lowest price in 30 days" on the product page from <code>price_history</code>	Claude
6	Tests: 5 price changes in one minute → one alert	Claude

Output of this phase

- Reviews, wishlist and price-drop alerts

Done when

- Lower a price → a wishlisted customer gets one alert (not five)

Explain it back

- Why do alerts come from a job every 10 minutes instead of at once?
- How do we know a reviewer really bought the product?

Skills you practise: Spring events, scheduled jobs, aggregate queries

Where your work goes: `backend\...\review`, `...\wishlist`, `...\notify`

Phase 10 - Smart search

Time: 1 week · **Group:** Shine · **Folder:** `Phase_10_Smart_Search\`

Goal: Standout #4: search that forgives typos, suggests while you type, and shows filters with counts.

Real-life example: A good shop assistant understands "iphnoe" means iPhone, and says "we have 12 Samsung and 8 Apple phones" before you ask.

Steps

#	Step	Who
1	Spike: Hibernate Search 8 with the Lucene backend on Spring Boot 4; index folder; rebuild on start	Claude
2	Map products: full-text fields (title, brand, description), keyword fields for facets	Claude
3	Search API: fuzzy match, filters, facets with counts, sort	Claude
4	Suggest API: suggestions after 2 letters	Claude
5	Frontend: search box with suggestions (debounce, keyboard), result page with facet counts	Claude
6	"Customers also bought" (SQL, cached 1 hour)	Claude
7	Decision: keep Lucene or fall back to MySQL FULLTEXT for hosting - written in <code>docs/decisions.md</code>	Together

Output of this phase

- Smart search with typo tolerance, suggestions and facets

Done when

- "iphnoe" finds iPhone
- Suggestions appear while typing

Explain it back

- What is an index, and why is search not a SQL LIKE query?
- How does fuzzy matching find "iphnoe"?

Skills you practise: Full-text search, Lucene, analyzers, facets, debounce

Where your work goes: `backend\...\search`, search components in `frontend`

Phase 11 - Testing & polish

Time: 1 week · **Group:** Always · **Folder:** `Phase_11_Testing_and_Polish\`

Goal: Portfolio quality: the whole demo story runs as an automated browser test, the code is well covered, safe, fast and pleasant on every screen.

Real-life example: Before the grand opening, an inspector walks through the whole shop as a customer, checks every fire exit, and the cleaners polish every glass door.

Steps

#	Step	Who
1	Playwright end-to-end test of the demo story (buy from 2 sellers, seller ships, customer returns) in CI	Claude
2	JaCoCo coverage about 80% on services; coverage badge	Claude
3	Security review: ownership tests, OWASP checklist, dependency check, secret scan of the Git history	Claude
4	Accessibility (axe), keyboard use, phone + dark mode on every page	Claude
5	Speed: lazy-loaded pages, Cloudinary image sizes; Lighthouse ≥ 90 on Home	Claude
6	Vitest for the main components	Claude
7	README draft	Claude

Output of this phase

- E2E tests in CI
- Coverage report
- Security and accessibility fixes

Done when

- CI green including the E2E test
- Lighthouse ≥ 90 on the home page

Explain it back

- What is the difference between a unit, an integration and an end-to-end test?
- What did the security review find, and how was it fixed?

Skills you practise: Playwright, coverage, security review, accessibility, performance

Where your work goes: `e2e\`, test folders, `README.md`

Phase 12 - Launch

Time: 1 week · **Group:** Always · **Folder:** `Phase_12_Launch\`

Goal: TriVoKo is live on the internet, with demo accounts, a strong README, a demo video and posts that a recruiter can see.

Real-life example: The ribbon is cut, the shop opens to the public, and an advertisement goes into the newspaper.

Steps

#	Step	Who
1	Choose hosting: free (Render + Aiven + Vercel) or small paid (Railway / VPS)	You
2	Production settings: Docker image, environment variables, prod profile checks	Claude
3	Deploy database, backend and frontend; Stripe webhook URL; real email (Brevo)	Claude
4	Demo accounts and sample data in production; demo login buttons	Claude
5	Uptime check (keeps free hosting awake during job hunting)	Claude
6	Final README: GIFs, architecture + ER diagrams, k6 graph, badges, how to run	Claude
7	3-minute demo video (English; Tamil optional)	Together
8	Resume bullet points and LinkedIn post	Together
9	Final project guide + viva Q&A document	Claude
10	Git tag v1.0	Claude

Output of this phase

- Live link
- README, demo video, resume bullets, LinkedIn post
- Final guide + viva Q&A

Done when

- A friend opens the link on a phone, buys, returns, and it all works

Explain it back

- Explain TriVoKo in 2 minutes as you would in an interview
- Which part are you most proud of, and why?

Skills you practise: Deployment, production configuration, documentation, presenting your work

Where your work goes: hosting dashboards, `README.md`, `00_Project_Documents\`